

RADIUS authentication and dynamic VLAN assignment for WPA2 Enterprise using SQLite in FreeRADIUS

2016-09-09

I recently bought a UniFI AP AC Pro [1] access point to replace my old useless AP. For obvious geeky reasons I wanted to use WPA2 Enterprise instead of WPA2 Personal. In that way, I can have different accounts for accessing my wireless network, which means I can easily revoke access to someone using my WiFi.

Using WPA2 Enterprise requires the use of a RADIUS server, so I have written down the steps I used to configure this together with RADIUS assigned VLANs, so that different accounts gets different VLANs. I have stored this configuration inside an SQLite database for simplicity. An interesting and possible future tweak may be to move to an LDAP server instead.

My RADIUS server will be running FreeBSD, if you use some other system, the paths may vary.

Start by installing the FreeRADIUS port. Remember to include support for SQLite when building.

```
# portmaster net/freeradius3
```

First of all, modify the `/usr/local/etc/raddb/clients.conf` file and include a new block for the RADIUS client (i.e. your access point) and select a good secret. You then need to enter this together with the RADIUS server's IP in your APs configuration.

```
client apname {  
    ipaddr      = my.ap.example.com  
    secret      = nogoodsecret  
    nas_type    = other  
}
```

Modify the configuration file so that an SQLite database is used as backend. Edit `/usr/local/etc/raddb/mods-available/sql` and modify the `driver` line to:

```
driver = "rlm_sql_sqlite"
```

In the same file, uncomment the following block and set the correct database filename.

```
sqlite {  
  
    # Path to the sqlite database  
    filename = "/usr/local/etc/raddb/freeradius.db"  
  
    # How long to wait for write locks on the database to be  
    # released (in ms) before giving up.  
    busy_timeout = 200  
  
    # If the file above does not exist and bootstrap is set  
    # a new database file will be created, and the SQL statements
```

```
# contained within the bootstrap file will be executed.  
bootstrap = "${modconfdir}/${!:.:name}/main/sqlite/schema.sql"  
}
```

Finally enable the `sql` module.

```
# cd /usr/local/etc/raddb/mods-enabled  
# ln -s ../mods-available/sql
```

Next step is to initialize the database schema and set the correct permissions.

```
# cd /usr/local/etc/raddb  
# sqlite3 freeradius.db < mods-config/sql/main/sqlite/schema.sql  
# chown freeradius:freeradius freeradius.db  
# chmod 660 freeradius.db
```

After this, populate the database with some user accounts and groups. I have created two different user groups, `family` and `friends`, which will be used later on to differentiate VLAN assignments. I have opted to store the passwords as an NT-hash, since that allows us to use EAP-PEAP-MSCHAPv2 as EAP method later on. This EAP method is widely supported on many platforms (including e.g. Windows). The other possibility would be to store the passwords in clear text, which allows us to use any EAP method. But for obvious security reasons that is just stupid.

Before inserting the password hashes in the database, we need to calculate them. This can be done by using the program `smbencrypt`.

```
$ smbencrypt password1 password2 password3
LM Hash          NT Hash
-----
E52CAC67419A9A2238F10713B629B565    5835048CE94AD0564E29A924A03510EF
E52CAC67419A9A22F96F275E1115B16F    E22E04519AA757D12F1219C4F31252F4
E52CAC67419A9A221B087C18752BDBEE    BD7DFBF29A93F93C63CB84790DA00E63
```

After this we insert them into the database.

```
INSERT INTO radusergroup (username,groupname) VALUES ('user1', 'family');
INSERT INTO radusergroup (username,groupname) VALUES ('user2', 'friends');
INSERT INTO radusergroup (username,groupname) VALUES ('user3', 'friends');

INSERT INTO radcheck (username,attribute,op,value)
VALUES ('user1','NT-Password',':','=','5835048CE94AD0564E29A924A03510EF');
INSERT INTO radcheck (username,attribute,op,value)
VALUES ('user2','NT-Password',':','=','E22E04519AA757D12F1219C4F31252F4');
INSERT INTO radcheck (username,attribute,op,value)
VALUES ('user3','NT-Password',':','=','BD7DFBF29A93F93C63CB84790DA00E63');
```

You may now start the RADIUS server. It should work for EAP authentication.

However, we're still using the example certificates which is suboptimal. Let's generate a new certificate signed by a CA. Just follow the instructions in `/usr/local/etc/raddb/certs/README`. The instructions are excellent. However, do note that if you change the password from `whatever` you also need to change it inside `passwords.mk`. If you changed the server key password, don't forget to change it inside

FreeRADIUS config as well. In `/usr/local/etc/raddb/mods-available/eap` change `private_key_password` to your password. After this, restart the RADIUS server and enjoy.

VLAN assignment

I want to assign a client to different VLANs depending on the groups above. If a user belongs to the `family` group, they should be assigned to VLAN 1, otherwise to VLAN 149. Note that to do this you need to update both UniFI controller to a recent version >5.0.0 *and* the AP's firmware. I initially forgot to upgrade the AP's firmware (it was still at 3.4.14.3413), which created the unfortunate situation that the option *Enable RADIUS assigned VLAN* was available in the controller, but it didn't have an actual effect on the AP due to lack of support in the firmware. I spent many hours debugging this... For a Ubiquiti access point the following attributes should be added to the RADIUS response [2].

```
Tunnel-Type = 13,  
Tunnel-Medium-Type = 6,  
Tunnel-Private-Group-Id = "149"    # <=== add your vlan id for each user.
```

To add the attributes above, add them to the SQLite database as below:

```
INSERT INTO radgroupreply (groupname, attribute, value, op)  
VALUES ('friends', 'Tunnel-Type', '13', ':=');  
INSERT INTO radgroupreply (groupname, attribute, value, op)  
VALUES ('friends', 'Tunnel-Medium-Type', '6', ':=');  
INSERT INTO radgroupreply (groupname, attribute, value, op)  
VALUES ('friends', 'Tunnel-Private-Group-Id', '149', ':=');
```

Since VLAN 1 is the default VLAN, I do not add entries for the `family` group (note that things actually will not work if you do!). Note that for the attributes to propagate to the final RADIUS `Access-Accept` used by the AP to do the VLAN assignment, you need to modify `/usr/local/etc/raddb/sites-available/inner-tunnel` and go to the section `post-auth` and uncomment the following two blocks so it looks like this:

```
#
# Instead of "use_tunneled_reply", uncomment the
# next two "update" blocks.
#
update {
    &outer.session-state: += &reply:
}

#
# These attributes are for the inner session only.
# They MUST NOT be sent in the outer reply.
#
# If you uncomment the previous block and leave
# this one commented out, WiFi WILL NOT WORK,
# because the client will get two MS-MPPE-keys
#
update outer.session-state {
    MS-MPPE-Encryption-Policy !* ANY
    MS-MPPE-Encryption-Types !* ANY
    MS-MPPE-Send-Key !* ANY
    MS-MPPE-Recv-Key !* ANY
    Message-Authenticator !* ANY
    EAP-Message !* ANY
```

```
Proxy-State !* ANY  
}
```

After this, everything should work flawlessly!

References

[1] <https://www.ubnt.com/unifi/unifi-ap-ac-pro/>

[2] <https://help.ubnt.com/hc/en-us/articles/219654087-UniFi-Using-VLANs-with-UniFi-Wireless-Routing-Switching-Hardware>

Comments, thoughts, or questions?

Write to my public inbox <https://lists.sr.ht/~zozs/public-inbox>